

Кросплатформне програмування

08

Інтерфейси та
абстрактні класи



Інтерфейси та абстрактні класи

Про що йтиметься:

- Абстрактні методи та абстрактні класи
- Знайомство з інтерфейсами
- Реалізація інтерфейсів у класі
- **Інтерфейсні змінні**
- Методи інтерфейсів із кодом за замовчуванням
- Розширення інтерфейсів
- Спадкування класів та реалізація інтерфейсів



Абстрактні методи та класи

- При описі методу в класі вказується тип результату, назва методу та список аргументів, а також тіло методу з програмним кодом. Але можна обмежитися оголошенням методу – вказати його сигнатуру, але не описувати тіло методу. Відповідний метод називається абстрактним
- Кожен абстрактний метод описується з ключовим словом **abstract**
- Клас, у якому є хоча б один абстрактний метод, також називається абстрактним
- Абстрактний клас описується з ключовим словом **abstract**
- Якщо клас просто описати з ключовим словом **abstract**, то він буде абстрактним, навіть якщо в ньому немає абстрактних методів
- На основі абстрактного класу не можна створити об'єкт. Зазвичай їх використовують при успадкуванні
- Можна оголосити об'єктну змінну абстрактного класу. Така змінна може посилатися на об'єкт будь-якого підкласу, створеного на основі даного абстрактного класу
- Якщо в підкласі, який створюється на основі абстрактного суперкласу, деякі абстрактні методи залишити не описаними, то такий підклас також буде абстрактним
- Якщо клас просто описати з ключовим словом **abstract**, то він буде абстрактним, навіть якщо в ньому немає абстрактних методів



Програма: абстрактні класи

Ми створюємо кілька класів, які у певному сенсі описують різні геометричні фігури (рівносторонній трикутник, квадрат та коло). Ми припускаємо наявність у геометричних фігур таких характеристик, як колір (розмальована фігура), лінійний розмір (для трикутника і квадрата довжина сторони, для кола радіус) і площа

У програмі описується абстрактний клас із назвою `ColoredFigure`, на основі якого шляхом успадкування створюються класи `Triangle` (клас для опису трикутника), `Square` (клас для опису квадрата) та `Circle` (клас для опису кола). У головному методі програми створюються об'єкти похідних класів та показано, як отримати доступ до об'єктів через об'єктні змінні підкласів та через об'єктну змінну абстрактного суперкласу



Приклад: використання абстрактного класу - 1

```
abstract class ColoredFigure{ // Абстрактний суперклас
    String color; // Текстове поле (колір)
    int size; // Цілочисленне поле (розмір)
    ColoredFigure(String clr,int s){ // Конструктор
        // Привласнення значень полям:
        color=clr;
        size = s;
    } // Метод для відображення інформації про об'єкт:
    void show() {
        // Колір та назва фігури:
        System.out.println("Фігура: "+color+" "+getName());
        // Характерний розмір:
        System.out.println("Характерний розмір (" +getSizeName()+"): "+size);
        // Площа:
        System.out.printf("Площа: %.3f\n", getArea());
        // Імпровізована "лінія" із "зірочок":
        String line="";
        for(int k=1;k<=30;k++){
            line+="*";
        }
        System.out.println(line);
    }
    // Метод результатом повертає назву для
    // параметра, що визначає характерний розмір фігури:
    String getSizeName(){
        return "сторона";
    }
    // Абстрактні методи:
    abstract String getName(); // Назва фігури
    abstract double getArea(); // Площа фігури
}
// Продовження на наступному слайді!
```



Приклад: використання абстрактного класу - 2

```
class Triangle extends ColoredFigure{ // Підклас (фігура - трикутник)
    Triangle (String clr, int s) { // Конструктор
        super(clr,s); // Виклик конструктора суперкласу
    }
    // Опис абстрактного методу із суперкласу, що повертає назву фігури:
    String getName(){
        return "трикутник";
    }
    // Опис абстрактного методу із суперкласу, що повертає площу фігури:
    double getArea(){
        double k=Math.sqrt(3)/4;
        return size*size*k;
    }
}

class Square extends ColoredFigure{ // Підклас (фігура - квадрат)
    Square (String clr, int s) { // Конструктор
        super(clr,s); // Виклик конструктора суперкласу
    }
    // Опис абстрактного методу із суперкласу,
    // повертає результатом назву фігури:
    String getName(){
        return "квадрат";
    }
    // Опис абстрактного методу із суперкласу,
    // повертає результатом площа фігури:
    double getArea(){
        double k=1;
        return size*size*k;
    }
}

// Продовження на наступному слайді!
```



Приклад: використання абстрактного класу - 3

```
// Підклас (фігура - коло):
class Circle extends ColoredFigure{
    // Конструктор:
    Circle (String clr, int s) {
        // Виклик конструктора суперкласу:
        super(clr,s);
    }
    // Опис абстрактного методу із суперкласу,
    // повертає результатом назву фігури:
    String getName(){
        return "коло";
    }
    // Опис абстрактного методу із суперкласу,
    // повертає результатом площа фігури:
    double getArea(){
        double k = Math.PI;
        return size*size*k;
    }
    // Перевизначення методу, що повертає результатом
    // назву для параметра, що визначає характерний
    // розмір фігури:
    String getSizeName(){
        return "радіус";
    }
}
// Продовження на наступному слайді!
```



Приклад: використання абстрактного класу - 4

```
// Головний клас:
class Demo{
    // Головний метод:
    public static void main(String[] args){
        // Створення об'єктів:
        Triangle T=new Triangle("червоний",2); // Трикутник
        Square S=new Square("чорний",3); // Квадрат
        Circle C=new Circle("жовтий",1); // Коло
        // Відображення інформації про об'єкти через
        // об'єктні змінні підкласів:
        System.out.println("Використання об'єктних змінних підкласу");
        T.show();
        S.show();
        C.show();
        // Об'єктна змінна абстрактного суперкласу:
        ColoredFigure F;
        // Відображення інформації про об'єкти через
        // Об'єктну змінну абстрактного суперкласу:
        System.out.println("Використання об'єктної змінної абстрактного суперкласу");
        F=T; // Трикутник
        F.show();
        F=S; // Квадрат
        F.show();
        F=C; // Коло
        F.show();
    }
}
```

Приклад: використання абстрактного класу - результат

Використання об'єктних змінних підкласів

Фігура: червоний трикутник

Характерний розмір (сторона): 2

Площа: 1,732

Фігура: чорний квадрат

Характерний розмір (сторона): 3

Площа: 9,000

Фігура: жовте коло

Характерний розмір (радіус): 1

Площа: 3,142

Використання об'єктної змінної абстрактного суперкласу

Фігура: червоний трикутник

Характерний розмір (сторона): 2

Площа: 1,732

Фігура: чорний квадрат

Характерний розмір (сторона): 3

Площа: 9,000

Фігура: жовте коло

Характерний розмір (радіус): 1

Площа: 3,142

Інтерфейси

- Інтерфейс нагадує абстрактний клас: інтерфейс містить оголошення методів та/або статичних констант
- У Java 8 інтерфейс може містити не тільки оголошення методів, але ще й опис методів так звані версії методів за замовчуванням
- Інтерфейси використовуються шляхом реалізації їх у класах
- Клас, який реалізує інтерфейс, повинен містити опис усіх методів, оголошених в інтерфейсі
- Один і той же клас може реалізувати одночасно кілька інтерфейсів



Реалізація інтерфейсу

- Інтерфейс описується так само, як і клас. Тільки замість ключового слова **class** використовується ключове слово **interface**
- Крім оголошення методів, в інтерфейсі можна оголошувати статичні константні поля. Такі поля оголошуються як звичайні поля зі значеннями, але автоматично інтерпретуються, ніби вони були описані з ключовими словами **static** і **final**
- Методи, оголошені в інтерфейсі, є **public**-методами (хоча при оголошенні методів в інтерфейсі ключове слово **public** не використовується)

Приклад описання інтерфейсу

```
interface MyInterface{  
    int NUMBER=100;  
    int getNumber(double x);  
    char getSymbol(int n);  
}
```

Приклад описання класу

```
class MyClass implements MyInterface{  
    public int getNumber(double x){  
        return (int)x;  
    }  
    public char getSymbol(int n){  
        return (char)('A'+n);  
    }  
}
```

Реалізація інтерфейсу - приклад

Результат програми:

Статична константа: 100
Ціле число: 12
Символ: D

```
// Інтерфейс:
interface MyInterface{
    //Статична константа:
    int NUMBER = 100;
    // Оголошення методів:
    int getNumber(double x);
    char getSymbol(int n);
}

// Клас реалізує інтерфейс:
class MyClass implements MyInterface{
    // Описання методів:
    public int getNumber(double x){
        return (int)x;
    }
    public char getSymbol(int n){
        return (char)('A'+n);
    }
}

// Головний клас:
class Demo{
    public static void main(String[] args){
        MyClass obj=new MyClass();
        System.out.println("Статична константа: "+MyClass.NUMBER);
        System.out.println("Ціле число: "+obj.getNumber(12.5));
        System.out.println("Символ: "+obj.getSymbol(3));
    }
}
```



Інтерфейсні змінні

- Інтерфейсна змінна - змінна, "тип" якої визначається назвою інтерфейсу
- Інтерфейсна змінна може посилатися на об'єкт класу, що реалізує цей інтерфейс

```
// Інтерфейс:
interface Base{
    // Оголошення методу:
    void show();
}
// Клас реалізує інтерфейс Base:
class Alpha implements Base{
    // Текстове поле:
    String word;
    // Конструктор:
    Alpha (String txt){
        word=txt;
    }
    // Опис методу з інтерфейсу:
    public void show(){
        System.out.println("Об'єкт класу Alpha");
        System.out.println("Текстове поле:" +word);
    }
}
```

```
// Клас реалізує інтерфейс Base:
class Bravo implements Base{
    // Цілочисленне поле:
    int number;
    // Конструктор:
    Bravo (int n){
        number=n;
    }
    // Опис методу з інтерфейсу:
    public void show(){
        System.out.println("Об'єкт класу Bravo");
        System.out.println("Цілочисленне поле: "+number);
    }
}
// Головний клас:
class Demo{
    public static void main(String[] args){
        // Інтерфейсна змінна:
        Base ref;
        // Об'єкт класу Alpha:
        ref=new Alpha("текст");
        ref.show();
        // Об'єкт класу Bravo:
        ref=new Bravo(123);
        ref.show();
    }
}
```

Результат:

Об'єкт класу Alpha
Текстове поле: текст
Об'єкт класу Bravo
Цілочисленне поле: 123

Методи з кодом за замовчуванням

- Метод в інтерфейсі можна не тільки оголосити, а й описати
- Якщо метод описаний в інтерфейсі, то в класі, що реалізує відповідний інтерфейс, метод можна не описувати
- Методи з кодом за замовчуванням описуються в інтерфейсі з ключовим словом **default**

```
// Інтерфейс з методом, що має код за замовчуванням:
interface Base{
    // Метод із кодом за замовчуванням:
    default void show(String txt){
        System.out.println("Інтерфейс Base: "+txt);
    }
    // Оголошення методу:
    void hello();
}

// Клас реалізує інтерфейс Base:
class Alpha implements Base{
    // Опис стандартного методу:
    public void hello(){
        System.out.println("Об'єкт класу Alpha");
    }
    // Опис методу з кодом за замовчуванням:
    public void show(String txt){
        System.out.println("Клас Alpha: "+txt);
    }
}
```

Результат:

Об'єкт класу Alpha
Клас Alpha: об'єктна змінна A
Клас Alpha: інтерфейсна змінна R
Об'єкт класу Bravo
Інтерфейс Base: об'єктна змінна B
Інтерфейс Base: інтерфейсна змінна R

```
class Bravo implements Base{ // Клас реалізує інтерфейс Base
    public void hello(){ // Опис звичайного методу
        System.out.println("Об'єкт класу Bravo");
    }
}

// Головний клас:
class Demo{
    public static void main(String[] args){
        Base R; // Інтерфейсна змінна
        Alpha A=new Alpha(); // Об'єкт класу Alpha
        // Виклик методів через об'єктну змінну:
        A.hello();
        A.show("об'єктна змінна A");
        // Інтерфейсної змінної присвоюється посилання об'єкт класу Alpha:
        R=A;
        // Виклик методу через інтерфейсну змінну:
        R.show("інтерфейсна змінна R");
        Bravo B=new Bravo(); // Об'єкт класу Bravo
        // Виклик методів через об'єктну змінну:
        B.hello();
        B.show("об'єктна змінна B");
        // Інтерфейсної змінної присвоюється посилання об'єкт класу Bravo:
        R=B;
        // Виклик методу через інтерфейсну змінну:
        R.show("інтерфейсна змінна R");
    }
}
```



Методи з кодом за замовчуванням

Виклик версії методу з інтерфейсу

Результат:

Метод із інтерфейсу First
Метод із інтерфейсу Second

```
// Перший інтерфейс:
interface First{
    default void hello(){
        System.out.println("Метод з інтерфейсу First");
    }
}

// Другий інтерфейс:
interface Second{
    default void hello(){
        System.out.println("Метод з інтерфейсу Second");
    }
}

// Клас реалізує два інтерфейси:
class MyClass implements First, Second{
    public void hello(){ // Опис методу
        // Виклик версії методу з інтерфейсу First:
        First.super.hello();
        // Виклик версії методу з інтерфейсу Second:
        Second.super.hello();
    }
}

// Головний клас:
class Demo{
    public static void main(String[] args){
        MyClass obj=new MyClass(); // Створення об'єкта
        // Виклик методу:
        obj.hello();
    }
}
```

Розширення інтерфейсів - 1

- Спадкування застосовується не тільки до класів, але і до інтерфейсів: один інтерфейс може успадковувати інший інтерфейс
- Спадкування інтерфейсів (називається розширенням інтерфейсу) реалізується так само, як і спадкування класів: в описі спадкового інтерфейсу після його імені вказується ключове слово **extends**, після якого вказується ім'я успадкованого інтерфейсу

```
// Інтерфейс для спадкування:  
interface First{  
    // Метод із кодом за замовчуванням:  
    default void alpha(){  
        System.out.println("Інтерфейс First: метод alpha()");  
    }  
    // Метод із кодом за замовчуванням:  
    default void bravo(){  
        System.out.println("Інтерфейс First: метод bravo()");  
    }  
    // Метод із кодом за замовчуванням:  
    default void charlie(){  
        System.out.println("Інтерфейс First: метод charlie()");  
    }  
    // Метод без коду за замовчуванням:  
    void delta ();  
} // Продовження на наступному слайді!
```



Розширення інтерфейсів - 2

```
// Інтерфейс-спадкоємець:
interface Second extends First{
    // Оголошення методу:
    void bravo();
    // Метод із кодом за замовчуванням:
    default void charlie(){
        System.out.println("Інтерфейс Second: метод charlie()");
    }
    // Оголошення методу:
    void echo();
}
// Клас реалізує інтерфейс:
class MyClass implements Second{
    // Опис методів:
    public void bravo(){
        System.out.println("Клас MyClass: метод bravo()");
    }
    public void delta(){
        System.out.println("Клас MyClass: метод delta()");
    }
    public void echo(){
        System.out.println("Клас MyClass: метод echo()");
    }
} // Продовження на наступному слайді!
```



Розширення інтерфейсів - 3

```
// Головний клас:  
class Demo{  
    public static void main(String[] args){  
        // Створення об'єкта:  
        MyClass obj=new MyClass();  
        // Виклик методів:  
        obj.alpha();  
        obj.bravo();  
        obj.charlie();  
        obj.delta();  
        obj.echo();  
    }  
}
```

Результат:

Інтерфейс First: метод alpha()
Клас MyClass: метод bravo()
Інтерфейс Second: метод charlie()
Клас MyClass: метод delta()
Клас MyClass: метод echo()



Розширення інтерфейсів - 4

Результат:

Інтерфейс First: метод alpha()
Інтерфейс Bravo: метод alpha()
Інтерфейс Bravo: метод bravo()
Клас MyClass: метод bravo()

```
interface First { // Перший інтерфейс
    default void alpha() { // Опис методу
        System.out.println("Інтерфейс First: метод alpha()");
    }
}

// Другий інтерфейс:
interface Second extends First {
    default void alpha() { // Опис методів
        First.super.alpha(); // Виклик версії методу з інтерфейсу First
        System.out.println("Інтерфейс Bravo: метод alpha()");
    }
    default void bravo() {
        System.out.println("Інтерфейс Bravo: метод bravo()");
    }
}

// Клас реалізує інтерфейс:
class MyClass implements Second {
    public void bravo() { // Опис методу
        Second.super.bravo(); // Виклик версії методу з інтерфейсу Second
        System.out.println("Клас MyClass: метод bravo()");
    }
}

// Головний клас:
class Demo {
    public static void main(String[] args) {
        MyClass obj = new MyClass(); // Створення об'єкта
        obj.alpha(); // Виклик методів
        obj.bravo();
    }
}
```



Спадкування класу та реалізація інтерфейсів - 1

Клас може успадковувати суперклас і одночасно реалізовувати кілька інтерфейсів

```
// Перший інтерфейс:
interface First{
    String getFirst(); // Оголошення методу
    default void show(){ // Опис методу
        System.out.println("Інтерфейс First: метод show()");
    }
}

// Другий інтерфейс:
interface Second{
    String getSecond(); // Оголошення методу
    default void show(){ // Опис методу
        System.out.println("Інтерфейс Second: метод show()");
    }
}

// Супер клас:
class Base{
    // Опис методу:
    String getBase(){
        return "Клас Base";
    }
    // Опис методу:
    void show() {
        System.out.println("Клас Base: метод show()");
    }
} // Продовження на наступному слайді!
```



Спадкування класу та реалізація інтерфейсів - 2

```
// Підклас успадковує суперклас і реалізує інтерфейси:
class MyClass extends Base implements First, Second{
    public String getFirst(){ // Опис методу
        return "Інтерфейс First";
    }
    // Опис методу:
    public String getSecond(){
        return "Інтерфейс Bravo";
    }
    // Опис методу:
    public void show(){
        System.out.println("Ми використовуємо:");
        System.out.println(getFirst());
        System.out.println(getSecond());
        System.out.println(getBase());
        First.super.show(); // Виклик версії методу з інтерфейсу First
        Second.super.show(); // Виклик версії методу з інтерфейсу Second
        super.show(); // Виклик версії методу із суперкласу Base
    }
}

// Головний клас:
class Demo{
    public static void main(String[] args){
        // Створення об'єкта підкласу:
        MyClass obj=new MyClass();
        // Виклик методу з об'єкта:
        obj.show();
    }
}
```

Результат:

Ми використовуємо:

Інтерфейс First

Інтерфейс Bravo

Клас Base

Інтерфейс First: метод show()

Інтерфейс Second: метод show()

Клас Base: метод show()

Дякую за увагу!

Запитання?